Nesne Yönelimli Programlama

Hafta 4: Diziler, Karakter Dizileri ve Dinamik Vektörler

Dr. Hüseyin Yüce

Marmara Üniversitesi Teknoloji Fakültesi

Mekatronik Mühendisliği Bölümü

Hafta 4: Teorik ve Kavramsal Hedefler

Bu hafta hedeflenen temel kavramsal kazanımlar:

- Dizi Mimarisi ve Bellek Düzeni:
  - Tek ve çok boyutlu dizilerin fiziki bellekte ardışıl yerleşimi (*Row-Major Order*)
  - Dizi indisleme matematiği ve bellek ofset hesaplama kuralları
- Bellek Güvenliği ve Sınır Denetimi:
  - Sınır aşımı (*Out-of-Bounds*) ve Tanımsız Davranış (*Undefined Behavior*) riskleri
  - Dizi-gösterici bozunması (*Array Decaying*) mekanizması
- Metin Temsili ve Karakter Dizileri:
  - C-tarzı karakter dizileri (`char[]`), null sonlandırıcı (`'\0'`) ve `<cstring>` riskleri
  - Modern `std::string` sınıfı, dinamik bellek yönetimi ve güvenli metotlar
- Dinamik Veri Koleksiyonları ve Algoritmalar:
  - Sabit `std::array` vs Dinamik `std::vector` konteynerleri
  - Temel arama (Lineer, İkili) ve sıralama (Kabarcık Sıralama) algoritmaları

> Dizi (Array): Aynı veri tipindeki sabit sayıda elemanın bellekte kesintisiz sıralandığı temel veri yapısı.

Hafta 4: Uygulamalı ve Mekatronik Hedefler

Laboratuvar oturumu ve gömülü sistem mimarisi kazanımları:

- Sensör Dizisi Taraması ve Veri İşleme:
  - 10 kanallı ADC sensör verilerinin tek boyutlu dizide toplanması
  - Min, max ve ortalama telemetri hesaplama döngüleri
- Sinyal Gürültü Filtreleme:
  - Mekatronik algılayıcılar için Kayan Ortalama Filtresi (*Moving Average Filter*) tasarımı
- Seri Haberleşme Metin Ayrıştırma (Parsing):
  - UART seri porttan gelen NMEA/GPS (`$GPGGA`) cümlelerinin `std::string` ile ayrıştırılması
- Dinamik Telemetri Kaydı:
  - `std::vector` ile bellek sızıntısız dinamik telemetri günlüğü ve eşik denetimi

ADC (Analog-to-Digital Converter): Sürekli analog gerilim sinyalini sayısal tamsayı değerine dönüştüren donanım birimi.

Motivasyon: Neden Diziler ve Veri Koleksiyonları?

Mekatronik sistemlerde onlarca sensör ve aktüatör verisi eş zamanlı işlenir:

- **Skaler Değişkenlerin Yetersizliği:** 10 kanallı bir ADC için `adc0, adc1, ..., adc9` tanımlamak kod tekrarına yol açar ve döngülerle işlenemez.
- **Önbellek Yerelliği (Cache Locality):** Diziler bellekte yan yana (ardışıl) yerleştiğinden CPU önbelleği (*L1/L2 cache*) verilere mikrosaniyeler içinde erişir.
- **Algoritmik Otomasyon:** Arama, sıralama, filtreleme ve istatistiksel hesaplamalar döngüler ve dizi indisleri sayesinde tek bir fonksiyona bağlanır.
- **Mühendislik Standartı:** Robotik haritalar, lidar nokta bulutları ve seri haberleşme paketleri dizi yapıları üzerinde modellenir.

> **Mühendislik İlkesi:** Aynı türden birden fazla veriyi tek tek değişkenler yerine homojen bir veri koleksiyonunda saklamak temel yazılım disiplinidir.

Tek Boyutlu Diziler: Tanım ve Temel Yapı

Dizi, aynı veri tipine sahip sabit sayıda elemanın bellekte ardışıl saklandığı yapıdır:

```
1  // [Veri Tipi] [Dizi Adı][Eleman Sayısı];
2  int adcDegerleri[5];        // 5 elemanlı tamsayı dizisi (ilklendirilmemiş)
3  double eklemAculari[3];     // 3 eksenli robot kol eklem açısı dizisi
```

- **Sabit Boyut Kuralı:** Klasik C++ dizilerinin boyutu **derleme zamanında** bilinmeli ve sabit (`const` / `constexpr`) olmalıdır.
- **İlklendirilmemiş Değer Tehlikesi:** Yerel (*stack*) dizi tanımlandığında elemanlar sıfırlanmaz; bellekteki artık (*çöp - garbage*) değerleri taşır.
- **Tip Güvenliği:** Bir dizi yalnızca tanımlandığı tipte veri tutabilir (`int` dizisine `string` konulamaz).

> **Stack (Yığın):** Fonksiyon çağrıları ve yerel değişkenler için derleyicinin otomatik ayırdığı hızlı bellek alanı.

Dizileri İlklendirme Yöntemleri

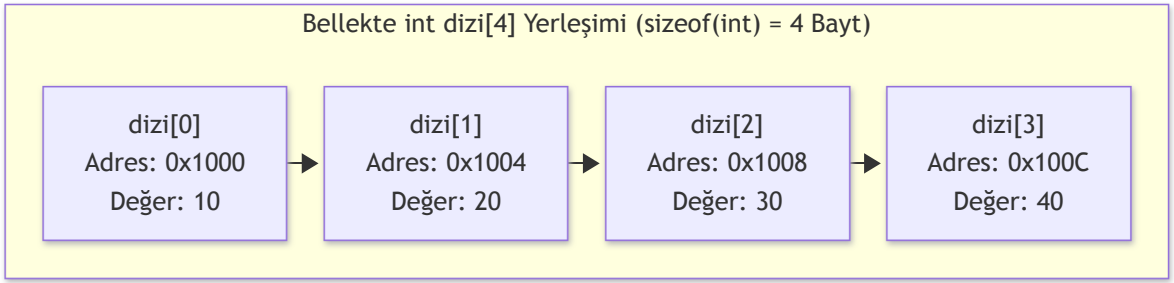
Diziler tanımlandığı anda süslü parantez `{}` ile ilklendirilebilir:

```
1  // 1. Tam İlklendirme:
2  int sayilar[5] = {10, 20, 30, 40, 50};
3
4  // 2. Kısmi İlklendirme: Belirtilmeyen elemanlar otomatik 0 olur!
5  int sensorler[5] = {12, 45}; // {12, 45, 0, 0, 0}
6
7  // 3. Boyut Belirtmeden İlklendirme: Derleyici eleman sayısını sayar (Boyut: 3)
8  double voltajlar[] = {3.3, 5.0, 12.0};
9
10 // 4. Sıfır ile İlklendirme: Tüm elemanlar 0 olur
11 int sayaclar[10] = {0};
```

Kritik Kural: `int a[5] = {1};` yapıldığında tüm elemanlar 1 **olmaz**! İlk eleman 1, geri kalan 4 eleman 0 olur.

Dizilerin Bellek Anatomisi ve Ardışıl Yerleşim

Diziler bellekte kesintisiz ve ardışıl bir blok halinde depolanır:



- Adres Formülü: $Adres(dizi[i]) = BaslangicAdresi + i \times sizeof(VeriTipi)$
- $\mathcal{O}(1)$ Erişim Hızı: İndis bilindiğinde formülle doğrudan adrese gidilir; aradaki elemanları gezmek gerekmez.
- 0-Tabanlı İndis: $i = 0$ başlangıç adresinin tam kendisidir ($0 \times sizeof$).

Dizi İndisleme ve Sınır Aşımı (Out-of-Bounds)

C++ dilinde performans gerekçesiyle çalışma zamanında dizi sınır denetimi yapılmaz:

```cpp
#include <iostream>

int main() {
    int sensorler[3] = {100, 200, 300}; // Geçerli indisler: 0, 1, 2

    std::cout << sensorler[0] << "\n";  // 100 (Geçerli)
    std::cout << sensorler[2] << "\n";  // 300 (Geçerli)

    // TEHLİKE: Sınır Aşımı (Out-of-Bounds)!
    std::cout << sensorler[5] << "\n";  // Tanımsız Davranış (Undefined Behavior)
    sensorler[5] = 999;                 // Bellek Bozulması (Stack Smashing)!
    return 0;
}
```

Tanımsız Davranış (UB): Sınır dışına yazmak programın aniden çökmesine, gizli hatalara ya da güvenlik açıklarına (*Buffer Overflow*) yol açar!

Sınır Denetimi: Hatalı vs Güvenli Yaklaşım

C++ dizilerinde sınır aşımını engelleme kuralı:

```cpp
int veriler[4] = {1, 2, 3, 4};
const int boyut = 4;

// 1. HATALI: i <= 4 kullanımı (veriler[4] sınır dışıdır, tanımsız davranış üretir!)
// for (int i = 0; i <= 4; ++i) { std::cout << veriler[i] << " "; }

// 2. GÜVENLİ: i < boyut kuralı (Sınırlar aşılmaz, güvenli okuma sağlanır)
for (int i = 0; i < boyut; ++i) {
    std::cout << veriler[i] << " ";
}
```

- N Elemanlı Dizi: Son geçerli indis daima $N - 1$'dir; döngü koşulu mutlaka `i < N` olmalıdır.
- Bellek Bozulması: Sınır aşımı komşu yerel değişkenleri veya fonksiyon çağrı çerçevesini bozar.
- Modern Alternatif: İndis hatasını önlemek için `std::array::at()` veya aralık tabanlı `for (: )` tercih edilmelidir.

Buffer Overflow: Bir bellek tamponunun sınırlarını aşarak bitişik bellek hücrelerine taşması güvenlik açığı.

Aralık Tabanlı For Döngüsü (Range-Based For)

C++11 ile dizileri sınır hatası olmadan gezmek için aralık tabanlı döngü sunulmuştur:

```cpp
#include <iostream>

int main() {
    double gerilimler[] = {3.3, 5.0, 12.0, 24.0};

    // 1. Salt Okuma (Kopyalama ile)
    for (double g : gerilimler) {
        std::cout << g << " V ";
    }
    std::cout << "\n";

    // 2. Referans ile Güncelleme (Bellek tasarrufu ve değer değiştirme)
    for (double& g : gerilimler) {
        g *= 1.1; // %10 gerilim artışı
    }
    return 0;
}
```

Avantaj: İndis takibi gerektirmez, sınır aşımı ihtimalini tamamen ortadan kaldırır.

Fonksiyonlara Dizi Aktarımı ve Dizi Bozunması

Diziler bir fonksiyona aktarıldığında dizi kimliğini kaybedip ilk elemanın işaretçisine dönüşür:

```cpp
#include <iostream>

// Derleyici için int dizi[] ile int* dizi tamamen AYNI ifadedir!
void analizEt(int dizi[]) {
    // sizeof(dizi) dizi boyutunu DEĞİL, gösterici boyutunu (8 bayt) verir!
    std::cout << "Fonksiyon ici sizeof: " << sizeof(dizi) << " bayt\n";
}

int main() {
    int sayilar[10] = {0};
    std::cout << "main ici sizeof: " << sizeof(sayilar) << " bayt\n"; // 40 bayt
    analizEt(sayilar); // 8 bayt (64-bit mimaride pointer boyutu)
    return 0;
}
```

Dizi Bozunması (Array Decaying): Dizi adı fonksiyona aktarılırken ilk elemanın bellek adresine (`int*`) indirgenir; eleman sayısı bilgisi kaybolur!

Fonksiyonlara Dizi Aktarımı Standartı

Dizi bozunması nedeniyle fonksiyonlara dizi ile birlikte **eleman sayısı da parametre** olarak verilmelidir:

```cpp
#include <iostream>

// Dizi üzerinde değişiklik yapılmayacaksa const kullanılmalıdır!
void diziYazdir(const int dizi[], int boyut) {
    for (int i = 0; i < boyut; ++i) {
        std::cout << dizi[i] << " ";
    }
    std::cout << "\n";
}

int main() {
    int telemetri[4] = {120, 145, 132, 160};
    const int boyut = sizeof(telemetri) / sizeof(telemetri[0]); // 16 / 4 = 4

    diziYazdir(telemetri, boyut);
    return 0;
}
```

Mühendislik Kuralı: Boyut parametresi aktarılmazsa fonksiyon dizinin nerede bittiğini bilemez ve sınır aşımı hatası oluşur.

Çok Boyutlu Diziler (2D Matrisler)

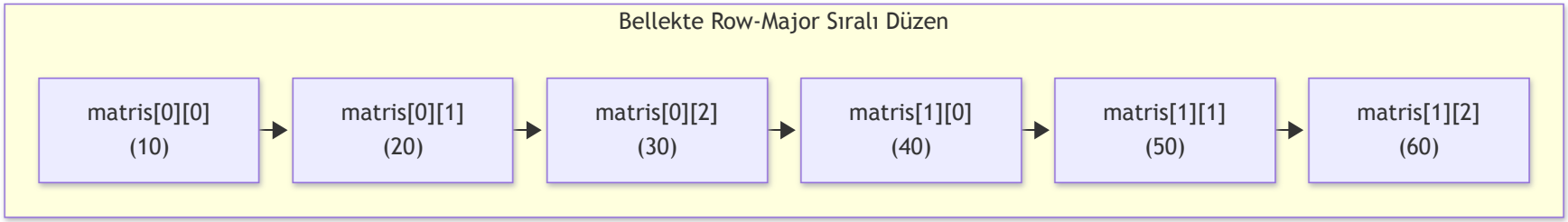
İki boyutlu diziler satır ve sütunlardan oluşan matrisleri modeller:

```
1  // [Tip] [Dizi Adı][Satır Sayısı][Sütun Sayısı];
2  int matris[2][3] = {
3      {10, 20, 30}, // Satır 0
4      {40, 50, 60}  // Satır 1
5  };
6
7  // Elemana Erişim: matris[satirIndis][sutunIndis]
8  std::cout << matris[0][1]; // 20
9  std::cout << matris[1][2]; // 60
```

- **Matris Temsili:** Mekatronikte robotiğin dönüşüm matrisleri (4×4), kamera görüntü matrisleri ve çok kanallı sensör tabloları için kullanılır.
- **İç İçe Döngüler:** 2D dizilerin taranması dışta satır, içte sütun döngüsüyle yapılır.

Bellekte Satır-Öncelikli Yerleşim (Row-Major Order)

Bilgisayar belleği 2 boyutlu değildir; 2D matris bellekte **tek boyutlu ardışıl** dizilir:



- **Satır-Öncelikli Kural:** Önce 0. satırın tüm sütunları, hemen ardından 1. satırın sütunları belleğe yazılır.
- **2D Adres Formülü:** $Adres(M[i][j]) = Baslangic + (i \times SutunSayisi + j) \times sizeof(T)$
- **Performans İpucu:** Satır satır ilerlemek CPU Cache dostudur; sütun sütun atlamak performansı düşürür.

Row-Major Order: Çok boyutlu dizilerin belleğe satır satır sıralı olarak yerleştirilmesi mimarisi.

2D Dizi Örneği: Çok Kanallı Sensör Tablosu

```cpp
#include <iostream>
#include <iomanip>

int main() {
    const int SENSOR_SAYISI = 3;
    const int ORNEK_SAYISI = 4;

    double sensorVerileri[SENSOR_SAYISI][ORNEK_SAYISI] = {
        {23.4, 23.6, 23.5, 23.8}, // Sensör 0: Sıcaklık
        {45.1, 45.0, 44.9, 45.3}, // Sensör 1: Nem
        {101.2, 101.3, 101.1, 101.2} // Sensör 2: Basınç
    };

    for (int i = 0; i < SENSOR_SAYISI; ++i) {
        std::cout << "Sensor " << i << ": ";
        for (int j = 0; j < ORNEK_SAYISI; ++j) {
            std::cout << std::fixed << std::setprecision(1)
                      << sensorVerileri[i][j] << " | ";
        }
```

Karakter Dizileri: C-Tarzı Diziler (C-Strings)

C dilinden miras kalan karakter dizileri, sonu `'\0'` (null karakter) ile biten `char` dizileridir:

```cpp
// 1. Açık Null Belirteçli Tanımlama:
char komut1[5] = {'S', 'T', 'O', 'P', '\0'};

// 2. Çift Tırnak ile Tanımlama: '\0' otomatik eklenir (Dizi boyutu: 5!)
char komut2[] = "STOP";

std::cout << komut2 << "\n"; // cout '\0' görene kadar ekrana basar!
```

- **Null Sonlandırıcı (`'\0'`):** Sayısal değeri `0` olan özel karakterdir. Metnin nerede bittiğini gösterir.
- **$N + 1$ Kuralı:** 4 karakterlik bir kelime saklamak için dizinin en az 5 elemanlı olması gerekir.
- **Null Unutulursa:** `std::cout` bellekte başka bir sıfır baytı bulana kadar rastgele bellek okur (güvenlik açığı!).

Null Sonlandırıcı (\0): Karakter dizisinin bittiği noktayı belirten sıfır baytlık özel sonlandırma karakteri.

C-String Kütüphanesi ve Riskleri (`<cstring>`)

Geleneksel C-string fonksiyonları sınır denetimi yapmadığı için güvenlik riski taşır:

```cpp
#include <iostream>
#include <cstring> // strlen, strcpy, strcmp, strcat

int main() {
    char str1[20] = "Marmara";
    char str2[] = "Mekatronik";

    // 1. strlen: Null karakter hariç uzunluk
    std::cout << "Uzunluk: " << std::strlen(str1) << "\n"; // 7

    // 2. strcmp: Eşitse 0 döner
    if (std::strcmp(str1, "Marmara") == 0) {
        std::cout << "Dizeler esit!\n";
    }

    // 3. TEHLİKE: strcpy sınır denetimi yapmaz!
    // char kucukDizi[4];
    // std::strcpy(kucukDizi, "CokUzunMetin"); // BUFFER OVERFLOW!
    return 0;
```

Modern C++: `std::string` Sınıfı

Modern C++, C-string'lerin bellek risklerini ortadan kaldıran `std::string` sınıfını sunar:

```cpp
#include <iostream>
#include <string> // std::string sınıfı için

int main() {
    std::string cihazAdi = "Motor_Surucu_A1";
    std::string durum = "AKTIF";

    // Doğrudan birleştirme (+)
    std::string telemetri = cihazAdi + " -> " + durum;
    std::cout << telemetri << "\n";

    // Boyut bilgisi nesnenin içinde tutulur
    std::cout << "Karakter sayisi: " << telemetri.length() << "\n";
    return 0;
}
```

Dinamik Bellek ve RAII: `std::string` kendi belleğini otomatik yönetir. Karakter eklendikçe boyutu otomatik genişler, kapsam bitince belleği temizler.

`std::string` Temel Metotları

Metot	İşlev ve Açıklama
`size()` / `length()`	Karakter sayısını döner ($\mathcal{O}(1)$ zaman).
`empty()`	Dize boş ise `true`, değilse `false` döner.
`clear()`	Tüm metni temizler, dizeyi boşaltır.
`c_str()`	C kütüphaneleriyle uyumlu `const char*` işaretçisi döndürür.
`operator[]`	Belirtilen indisteki karaktere erişir (sınır denetimsiz).
`at(index)`	Sınır kontrollü erişim sağlar (aşımda `std::out_of_range` fırlatır).
`push_back(ch)`	Metnin sonuna tek bir karakter ekler.
`pop_back()`	Metnin sonundaki karakteri siler.

`std::string` Metin Arama ve Parçalama

Telemetri paketlerini ayrıştırmak için `find` ve `substr` metotları kullanılır:

```cpp
#include <iostream>
#include <string>

int main() {
    std::string paket = "CMD:HIZ=1500;STATUS=OK";

    // 1. Metin Arama: Belirtilen ifadenin başlangıç indisini bulur
    size_t pos = paket.find("HIZ=");
    if (pos != std::string::npos) {
        // 2. Alt Metin Alma (substr: başlangıç, uzunluk)
        std::string deger = paket.substr(pos + 4, 4); // "1500"
        std::cout << "Bulunan Hiz: " << deger << " RPM\n";
    }

    // 3. Karşılaştırma doğrudan == operatörü ile yapılır
    if (paket.substr(17) == "STATUS=OK") {
        std::cout << "Sistem durumu basarili.\n";
    }
    return 0;
```

Sabit Boyutlu Modern Alternatif: `std::array`

C++11 `<array>` kütüphanesi, C dizilerinin hızını STL güvenliği ile birleştirir:

```cpp
#include <iostream>
#include <array>

int main() {
    // std::array<Tip, Boyut> adi;
    std::array<int, 3> motorAkimlari = {12, 14, 11};

    // 1. Boyut bilgisi nesneyle birlikte taşınır
    std::cout << "Eleman sayisi: " << motorAkimlari.size() << "\n";

    // 2. Güvenli erişim: at() fonksiyonu sınır aşımını yakalar
    try {
        std::cout << motorAkimlari.at(5); // HATA: out_of_range istisnası!
    } catch (const std::out_of_range& e) {
        std::cout << "Yakalama: Gecersiz dizi indisi!\n";
    }
    return 0;
}
```

- Sıfır Maliyet (Zero-Cost Abstraction): `std::array` bellek olarak klasik C dizisiyle aynı boyuttadır (Stack üzerinde saklanır), ancak

Dinamik Boyutlu Diziler: `std::vector`'e Giriş

Eleman sayısı önceden bilinmeyen veya çalışma zamanında değişen veriler için `std::vector` kullanılır:

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> telemetri; // Boş dinamik dizi

    // Sonuna dinamik eleman ekleme: push_back()
    telemetri.push_back(100);
    telemetri.push_back(200);
    telemetri.push_back(300);

    std::cout << "Boyut: " << telemetri.size() << "\n"; // 3
    std::cout << "Son eleman: " << telemetri.back() << "\n"; // 300

    // Son elemanı silme: pop_back()
    telemetri.pop_back();
    std::cout << "Yeni boyut: " << telemetri.size() << "\n"; // 2
    return 0;
}
```

`std::vector` Bellek Büyüme Mekanizması

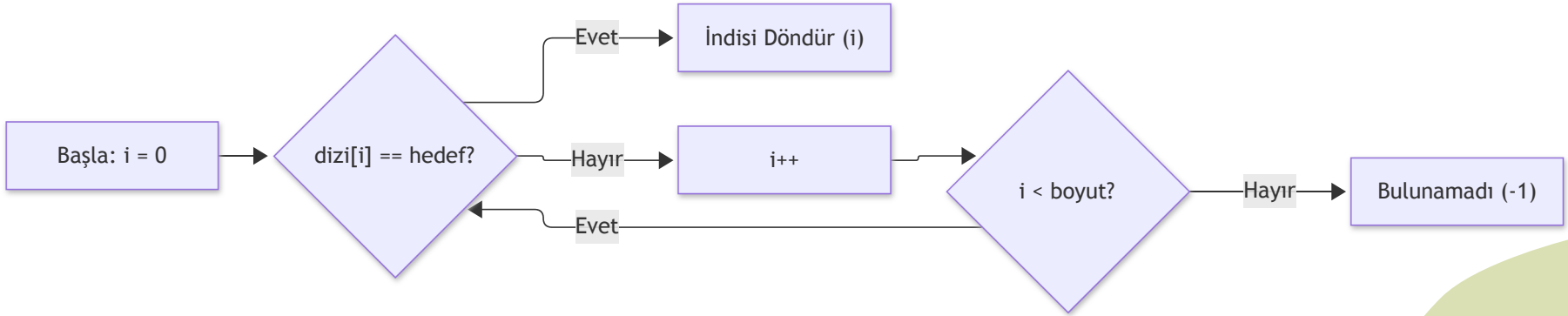
Vektör elemanları dinamik bellekte (*Heap*) ardışıl saklanır:

- `size()` : Vektörün içinde o anda bulunan gerçek eleman sayısı.
- `capacity()` : Vektörün yeniden bellek ayırmadan tutabileceği maksimum eleman sayısı.
- **Geometrik Büyüme:** Kapasite dolduğunda vektör arka planda $2\times$ (veya $1.5\times$) daha büyük yeni bir bellek bloğu açar, eski elemanları taşır ve eski belleği serbest bırakır.

```
1  std::vector<int> v;
2  for (int i = 0; i < 5; ++i) {
3      v.push_back(i);
4      std::cout << "Eleman: " << v.size() << " | Kapasite: " << v.capacity() << "\n";
5  }
6  // Çıktı Kapasiteleri: 1 -> 2 -> 4 -> 4 -> 8 (Dinamik genişleme)
```

Algoritma 1: Lineer Arama (Linear Search)

Sırasız bir veri kümesinde aranan elemanı bulmak için baştan sona tek tek karşılaştırma yapılır:



- **Önkoşul:** Dizinin sıralı olmasına gerek yoktur.
- **Zaman Karmaşıklığı:** $\mathcal{O}(n)$ — En kötü durumda tüm dizi taranır.
- **Kullanım:** Küçük boyutlu veya sürekli güncellenen sırasız mekatronik sensör listeleri.

Lineer Arama: Baştan sona sırayla kontrol eden $\mathcal{O}(n)$ karmaşıklıklı temel arama algoritması.

Lineer Arama C++ İmplementasyonu

```cpp
#include <iostream>

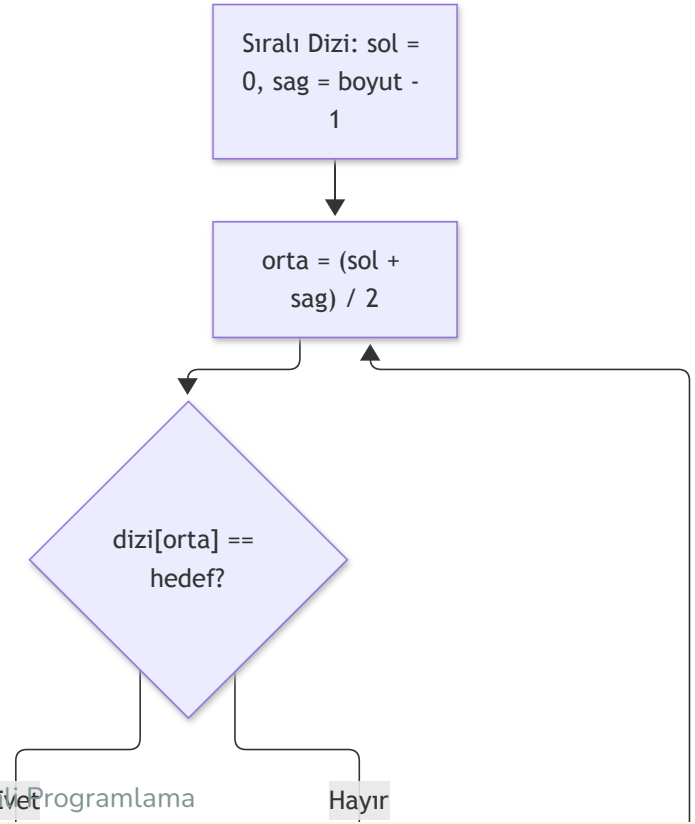
// Bulursa elemanın indisini, bulamazsa -1 döndürür
int lineerArama(const int dizi[], int boyut, int aranan) {
    for (int i = 0; i < boyut; ++i) {
        if (dizi[i] == aranan) {
            return i; // Bulundu
        }
    }
    return -1; // Bulunamadı
}

int main() {
    int hataKodlari[] = {101, 204, 404, 500, 502};
    const int boyut = sizeof(hataKodlari) / sizeof(hataKodlari[0]);

    int sonuc = lineerArama(hataKodlari, boyut, 404);
    if (sonuc != -1) {
        std::cout << "Hata kodu bulundu! Indis: " << sonuc << "\n";
```

Algoritma 2: İkili Arama (Binary Search)

Sıralı dizilerde arama uzayını her adımda yarıya indiren son derece hızlı bir algoritmadır:



İkili Arama C++ İmplementasyonu

```cpp
#include <iostream>

int ikiliArama(const int dizi[], int boyut, int hedef) {
    int sol = 0;
    int sag = boyut - 1;

    while (sol <= sag) {
        int orta = sol + (sag - sol) / 2; // Taşmayı önleyen orta nokta

        if (dizi[orta] == hedef) return orta; // Bulundu
        if (dizi[orta] < hedef) sol = orta + 1; // Hedef sağ tarafta
        else sag = orta - 1;                    // Hedef sol tarafta
    }
    return -1; // Bulunamadı
}

int main() {
    int siraliIdler[] = {10, 25, 34, 48, 56, 72, 89, 95};
    int n = sizeof(siraliIdler) / sizeof(siraliIdler[0]);
```

Algoritma 3: Kabarcık Sıralama (Bubble Sort)

Dizideki ardışıl elemanları ikili karşılaştırıp yanlış sıradakileri takas eden temel sıralama algoritmasıdır:

- **Çalışma Mantığı:** Her geçişte dizinin en büyük elemanı dizi sonuna doğru bir "kabarcık" gibi yükselir.
- **Zaman Karmaşıklığı:**
  - En Kötü / Ortalama: $\mathcal{O}(n^2)$
  - En İyi (Zaten sıralı ise optimize edilmiş): $\mathcal{O}(n)$
- **Takas Mekanizması:** İki değişkenin yer değiştirmesi `std::swap` ile gerçekleştirilir.

> **Mekatronik Uygulaması:** Sensör değerlerini küçükten büyüğe sıralayarak medyan (ortanca) gürültü filtresi oluşturmakta kullanılır.

Kabarcık Sıralama C++ İmplementasyonu

```cpp
#include <iostream>
#include <utility> // std::swap

void kabarcikSirala(int dizi[], int boyut) {
    for (int i = 0; i < boyut - 1; ++i) {
        bool takasOlduMu = false;
        for (int j = 0; j < boyut - i - 1; ++j) {
            if (dizi[j] > dizi[j + 1]) {
                std::swap(dizi[j], dizi[j + 1]);
                takasOlduMu = true;
            }
        }
        // Eğer hiçbir takas yapılmadıysa dizi zaten sıralıdır (Erken çıkış)
        if (!takasOlduMu) break;
    }
}

int main() {
    int sensorHam[] = {64, 34, 25, 12, 22, 11, 90};
```

Mekatronik Vaka Analizi 1: 10 Kanallı ADC Taraması

Problem: Bir endüstriyel robot denetleyicisinde 10 kanallı 10-bit ADC'den ham veri okunmaktadır. Sistem; minimum gerilimi, maksimum gerilimi ve kanalların ortalamasını hesaplamalı, kritik gerilim eşiğini (4.5 V) aşan kanalları raporlamalıdır:

- **Donanım Özellikleri:** 10-bit ADC çözünürlüğü ($[0, 1023]$), Referans gerilim: $5.0\ \mathrm{V}$.
- **Fiziksel Dönüşüm:** $V_{olculen} = \frac{ADC_{ham} \times 5.0}{1023.0}$
- **Veri Yapısı:** 10 elemanlı `double gerilimler[10]` dizisi.

ADC Sensör Tarama Kodu (İmplementasyon)

```cpp
#include <iostream>
#include <iomanip>

int main() {
    const int KANAL_SAYISI = 10;
    int hamAdc[KANAL_SAYISI] = {512, 102, 850, 950, 420, 1010, 310, 620, 780, 205};
    double gerilimler[KANAL_SAYISI];
    double toplam = 0.0, maxV = -1.0, minV = 999.0;

    for (int i = 0; i < KANAL_SAYISI; ++i) {
        gerilimler[i] = (hamAdc[i] * 5.0) / 1023.0; // Gerilim dönüşümü
        toplam += gerilimler[i];
        if (gerilimler[i] > maxV) maxV = gerilimler[i];
        if (gerilimler[i] < minV) minV = gerilimler[i];

        if (gerilimler[i] > 4.5) {
            std::cout << "[ALARM] Kanal " << i << " esik asimi: "
                      << gerilimler[i] << " V\n";
        }
```

Mekatronik Vaka Analizi 2: Kayan Ortalama Filtresi

Sensörlerden gelen yüksek frekanslı beyaz gürültüyü engellemek için N elemanlı kayan pencere kullanılır:



- Filtre Formülü: $y[k] = \frac{1}{N} \sum_{i=0}^{N-1} x[k-i]$
- **Mekatronik Önemi:** İvmeölçer, mesafe sensörü veya sıcaklık problarındaki anlık elektriksel parazitleri sönümler.
- **Veri Yapısı:** Sabit boyutlu dizi ve dairesel indis göstericisi.

> **Kayan Ortalama:** Gürültülü sensör sinyallerini yumuşatmak için son N örneğin ortalamasını alan dijital filtre.

Kayan Ortalama Filtresi Kodu

```cpp
#include <iostream>

class KayanOrtalamaFiltresi {
private:
    static const int PENCERE = 5;
    double tampon[PENCERE] = {0.0};
    int indis = 0;
    int elemanSayisi = 0;
public:
    double filtrele(double yeniVeri) {
        tampon[indis] = yeniVeri;
        indis = (indis + 1) % PENCERE; // Dairesel indeks
        if (elemanSayisi < PENCERE) elemanSayisi++;

        double toplam = 0.0;
        for (int i = 0; i < elemanSayisi; ++i) toplam += tampon[i];
        return toplam / elemanSayisi;
    }
};
```

Mekatronik Vaka Analizi 3: UART GPS NMEA Ayrıştırma

Otonom mobil robot GPS alıcısından UART üzerinden virgülle ayrılmış NMEA cümleleri okur:

```
$GPGGA,123519,4101.45,N,02858.32,E,1,08,0.9,54.2,M*42
```

- Paket Alanları:
  - İndis 0: Başlık (`$GPGGA`)
  - İndis 1: Zaman damgası (`123519` -> 12:35:19)
  - İndis 2-3: Enlem ve Yön (`4101.45, N`)
  - İndis 4-5: Boylam ve Yön (`02858.32, E`)
- Çözüm Algoritması: `std::string` ve virgül karakterlerini (`','`) bulan `find` metodu ile alanları parçalamak.

> **NMEA 0183:** Küresel Konumlandırma (GPS) alıcıları ve denizcilik cihazlarında kullanılan evrensel seri veri protokolü.

GPS NMEA Ayrıştırma Kodu

```cpp
#include <iostream>
#include <string>
#include <vector>

std::vector<std::string> virgulParcala(const std::string& cumle) {
    std::vector<std::string> parcalar;
    size_t baslangic = 0;
    size_t virgul = cumle.find(',');

    while (virgul != std::string::npos) {
        parcalar.push_back(cumle.substr(baslangic, virgul - baslangic));
        baslangic = virgul + 1;
        virgul = cumle.find(',', baslangic);
    }
    parcalar.push_back(cumle.substr(baslangic)); // Son eleman
    return parcalar;
}

int main() {
```

Mekatronik Vaka Analizi 4: `std::vector` ile Telemetri

Dinamik telemetri günlüğü: Robot çalıştığı sürece gelen sensör verilerini vektöre yazar ve istatistik üretir:

```cpp
#include <iostream>
#include <vector>
#include <numeric> // std::accumulate

int main() {
    std::vector<double> motorSicaklikLog;

    // Çalışma anında dinamik veri akışı
    motorSicaklikLog.push_back(42.5);
    motorSicaklikLog.push_back(44.1);
    motorSicaklikLog.push_back(48.7);
    motorSicaklikLog.push_back(53.2);

    std::cout << "Toplam Kayit Sayisi: " << motorSicaklikLog.size() << "\n";

    // Vektör elemanlarını listeleme
    for (size_t i = 0; i < motorSicaklikLog.size(); ++i) {
        std::cout << "Ornek [" << i << "]: " << motorSicaklikLog[i] << " C\n";
```

Hangi Konteyneri Ne Zaman Seçmeliyiz?

Mühendislik kararlarında doğru veri koleksiyonunu seçme kriterleri:

Özellik	C-Tarzı Dizi (`T[]`)	`std::array<T, N>`	`std::vector<T>`
Boyut	Derleme zamanı (Sabit)	Derleme zamanı (Sabit)	Çalışma zamanı (Dinamik)
Bellek Yeri	Stack / Statik	Stack	Heap (Dinamik Öbek)
Boyutunu Bilir mi?	❌ Hayır (Decaying)	✅ Evet (`size()`)	✅ Evet (`size()`)
Kopyalama Desteği	❌ Manuel (`for`)	✅ Doğrudan (`=`)	✅ Doğrudan (`=`)
Sınır Kontrolü	❌ Yok	✅ `at()` ile var	✅ `at()` ile var
Bellek Maliyeti	Sıfır ek yük	Sıfır ek yük	Küçük kontrol başlığı
Önerilen Kullanım	C donanım kütüphaneleri	Sabit boyutlu gömülü tampon	Genel amaçlı dinamik veri

Tasarım ve Uygulama Notları (Mühendislik İlkeleri)

Dizi ve veri koleksiyonlarında dikkat edilmesi gereken kritik yazılım kuralları:

- 1. Dizi Sınırını 1 Fazla Aşmak: N elemanlı dizinin son indisi $N-1$'dir; `for(int i=0; i<=N; ++i)` yazmak tanımsız davranış üretir.
- 2. `sizeof` ile Dizi Boyutu Bulma Hatası: Fonksiyona parametre giren `int dizi[]` aslında bir işaretçidir (`int*`). Fonksiyon içinde `sizeof(dizi)` eleman sayısını vermez.
- 3. C-String Null Karakterini Unutmak: `char s[4] = "kedi";` sığmaz; 4 harf + `'\0'` = en az 5 bayt gereklidir.
- 4. Dizileri Birbirine Doğrudan Atamak: `dizi2 = dizi1;` derleme hatasıdır; döngü veya `std::copy` kullanılmalıdır.
- 5. İkili Arama Öncesi Sıralamayı Unutmak: İkili arama yalnızca sıralı dizilerde çalışır; sırasız dizide ikili arama hatalı sonuç üretir.

Laboratuvar İçi Pratik Alıştırmalar

Laboratuvar oturumunda uygulanacak pratik mühendislik problemleri:

- Alıştırma 1 (Dairesel Tampon / Circular Buffer):
  - Sabit 8 elemanlı bir `std::array` kullanarak taşma yapmayan dairesel bir sensör kuyruğu tasarlayınız. Yeni veri geldikçe en eski verinin üzerine yazılmalı ve son 8 verinin hareketli ortalaması hesaplanmalıdır.
- Alıştırma 2 (Robot Hata Paketi Ayrıştırıcı):
  - UART üzerinden gelen `"ERR:DEV=ROBOT1,CODE=404,SEV=CRITICAL"` metin formatındaki paketi `std::string` metotları (`find` , `substr`) ile ayrıştırıp yapılandırılmış `HataKaydi` nesnesine dönüştüren C++ programını kodlayınız.
- Alıştırma 3 (Sensör Verisi Medyan Filtresi):
  - 7 kanallı ADC gürültü verisini Kabarcık Sıralama (*Bubble Sort*) ile küçükten büyüğe sıralayıp tam ortadaki medyan değeri döndüren fonksiyonu yazınız.

Medyan Filtresi: Aykırı uç değerleri (gürültü piklerini) eleyerek ortanca değeri seçen doğrusal olmayan dijital filtre.

Hafta 4: Özet ve Mühendislik Kazanımları

Bu derste ele alınan temel ilkeler:

- **Ardışıl Bellek:** Dizilerin bellekte kesintisiz sıralandığını, $Adres = Baslangic + i \times sizeof(T)$ formülüyle $\mathcal{O}(1)$ sürede erişildiğini öğrendik.
- **Güvenlik ve Modern Alternatifler:** C-dizilerinin sınır denetimi olmaması riskine karşı modern `std::array` ve dinamik `std::vector` kullanımının üstünlüklerini inceledik.
- **Karakter Dizileri:** C-string'lerin `'\0'` açığını ve modern `std::string` sınıfının zengin metin işleme yeteneklerini kavradık.
- **Algoritmik Temeller:** Lineer Arama ($\mathcal{O}(n)$), İkili Arama ($\mathcal{O}(\log n)$) ve Kabarcık Sıralama ($\mathcal{O}(n^2)$) algoritmalarını mekatronik telemetri senaryolarında modelledik.
- **Gelecek Hafta:** Hafta 5 - Pointerlar, Gösterici Aritmetiği ve Dinamik Bellek Yönetimi (`new` / `delete`).

Kaynaklar ve İletişim

- Ders Deposu: github.com/marmara-mekatronik/MRM3049
- **Kaynak Kitap:** Deitel & Deitel, *C++ How to Program* (Bölüm 7: Arrays and Vectors)
- **Kaynak Kitap:** D.S. Malik, *Data Structures Using C++* (Bölüm 1 & 3: Arrays & Strings)
- **İletişim:** Dr. Hüseyin Yüce (Marmara Üniversitesi Teknoloji Fakültesi / Mekatronik Mühendisliği)

Nesne Yönelimli Programlama • Hafta 4: Diziler, Karakter Dizileri ve Dinamik Vektörler